

MEMBER 3: BACKEND DEVELOPER - VIBE CODING ROADMAP

Smart India Hackathon 2026 | Team ILLUMINATI (ID: 27113) | PS: SIH1642

Primary Folder: **backend/** | Dedicated Branch: **feature/fastapi-backend** | Deliverables: **8 Files**

PHASE 1: FASTAPI CORE & PYDANTIC V2 SCHEMAS

Prompt 3.1

Create backend/config.py using pydantic-settings. Define Settings class with PROJECT_NAME='AeroSense-AI', API_V1_STR='/api/v1', CORS_ORIGINS=['http://localhost:5173', 'http://localhost:3000'], and DATABASE_URL='sqlite:///./weather_anomaly.db'.

Prompt 3.2

Create backend/schemas/weather.py. Define Pydantic v2 PredictionRequest schema with fields: location: str, month: Optional[int], temperature: float, relative_humidity: float, pressure: float, wind_speed: float, rainfall: float with physical bounds validators.

Prompt 3.3

In backend/schemas/weather.py, define ContributorItem schema (feature, contribution_pct, observed, expected, unit, direction).

Prompt 3.4

In backend/schemas/weather.py, define PredictionResponse schema (location, timestamp, is_anomaly, anomaly_score, severity, anomaly_type, contributors, explanation).

Prompt 3.5

In backend/schemas/weather.py, define schemas for LocationInfo (city, lat, lon, state, current_severity, current_score) and HistoricalDataPoint (timestamp, observed, expected_normal, upper_bound, lower_bound).

Prompt 3.6

Create backend/main.py. Initialize FastAPI application with title 'AeroSense-AI Weather Anomaly Platform', CORS middleware with allow_origins=['*'], allow_methods=['*'], and a root welcome endpoint.

PHASE 2: DATABASE LAYER (SQLALCHEMY)

Prompt 3.7

Create backend/database/connection.py. Set up SQLAlchemy engine, sessionmaker, and Base = declarative_base(). Use SQLite by default for zero-setup local dev with PostgreSQL support via DATABASE_URL.

Prompt 3.8

Create backend/database/models.py. Define WeatherObservationModel table (id, location, timestamp, temperature, relative_humidity, pressure, wind_speed, rainfall, created_at).

MEMBER 3: BACKEND DEVELOPER - VIBE CODING ROADMAP

Smart India Hackathon 2026 | Team ILLUMINATI (ID: 27113) | PS: SIH1642

Primary Folder: **backend/** | Dedicated Branch: **feature/fastapi-backend** | Deliverables: 8 Files

Prompt 3.9

In backend/database/models.py, define AnomalyEventModel table (id, location, timestamp, anomaly_score, severity, anomaly_type, contributors_json, explanation).

Prompt 3.10

In backend/database/connection.py, add an init_db() function creating all tables on application startup.

PHASE 3: SERVICE LAYER & GRACEFUL MOCK FALLBACKS

Prompt 3.11

Create backend/services/model_loader.py. Implement a singleton loader that loads ml/models/isolation_forest.joblib, scaler.joblib, and data/processed/baseline_statistics.json. If files don't exist yet, provide a graceful rule-based mock fallback so API works immediately for frontend testing!

Prompt 3.12

Create backend/services/prediction_service.py. Implement predict_weather_anomaly(request: PredictionRequest) -> PredictionResponse. Look up baselines, compute deviations, call model_loader, and format response.

Prompt 3.13

Create backend/services/explanation_service.py. Implement logic to calculate feature percentage contributions and construct a human-readable diagnostic sentence.

PHASE 4: REST API ENDPOINTS

Prompt 3.14

Create backend/routes/health.py with GET /health returning status 'ok', model loaded status, database connectivity, and version.

Prompt 3.15

Create backend/routes/locations.py with GET /locations returning all 10 monitored Indian cities with lat, lon, elevation, current anomaly score, and active severity level.

Prompt 3.16

Create backend/routes/weather.py with GET /weather/{location} returning latest weather observation for given city alongside expected monthly normals.

Prompt 3.17

Create backend/routes/anomalies.py with GET /anomalies (20 most recent national anomalies) and GET /anomalies/{location} (city history).

MEMBER 3: BACKEND DEVELOPER - VIBE CODING ROADMAP

Smart India Hackathon 2026 | Team ILLUMINATI (ID: 27113) | PS: SIH1642

Primary Folder: **backend/** | Dedicated Branch: **feature/fastapi-backend** | Deliverables: **8 Files**

Prompt 3.18

Create backend/routes/history.py with GET /history/{location} taking query params variable ('temperature') and days (30). Return observed vs expected baseline corridors for charts.

Prompt 3.19

Create backend/routes/predictions.py with POST /predict. Validate payload, invoke prediction_service, log to AnomalyEventModel if anomalous, and return PredictionResponse in <15ms.

Prompt 3.20

In backend/main.py, include all routers under /api/v1. Add custom exception handlers for 400 Bad Request and 404 Not Found. Verify uvicorn backend.main:app --reload loads Swagger at <http://localhost:8000/docs>.